

Master All 20 Agentic AI Design Patterns

A Complete Guide to Production-Ready AI Agent Architectures

Comprehensive Technical Report

Designed for AI Engineers & Architects

2025 Edition

Table of Contents

1. Introduction to Agentic AI Design Patterns	
2. Pattern 1: Prompt Chaining	
3. Pattern 2: Routing	
4. Pattern 3: Parallelization	
5. Pattern 4: Reflection	
6. Pattern 5: Tool Use	
7. Pattern 6: Planning	
8. Pattern 7: Multi-Agent Collaboration	
9. Pattern 8: Memory Management	
10. Pattern 9: Learning & Adaptation	
11. Pattern 10: Goal Setting & Monitoring	
12. Pattern 11: Exception Handling & Recovery	
13. Pattern 12: Human-in-the-Loop	
14. Pattern 13: Retrieval (RAG)	
15. Pattern 14: Inter-Agent Communication	
16. Pattern 15: Resource-Aware Optimization	
17. Pattern 16: Reasoning Techniques	
18. Pattern 17: Evaluation & Monitoring	
19. Pattern 18: Guardrails & Safety	
20. Pattern 19: Prioritization	
21. Pattern 20: Exploration & Discovery	
22. Conclusion	
23. References	

Introduction to Agentic AI Design Patterns

As Large Language Models (LLMs) evolve from simple chatbots to autonomous agents capable of executing complex workflows, the need for robust engineering architectures has become paramount. "Agentic AI Design Patterns" represent the crystallized wisdom of the AI engineering community—standardized, proven approaches to solving common problems in building autonomous systems.

Building a prototype agent is easy; building a production-ready agent that is reliable, cost-effective, and safe is exponentially harder. Without structured patterns, developers often create "spaghetti code" workflows that are brittle, hard to debug, and prone to hallucinations. These patterns provide the scaffolding necessary to move from experimental notebooks to enterprise-grade deployments.

This guide categorizes 20 essential patterns into a cohesive framework. These patterns span the entire lifecycle of an agent's operation: from how it receives and processes a task (Routing, Prompt Chaining), to how it thinks (Reasoning, Reflection), acts (Tool Use, Collaboration), and improves (Learning, Evaluation). By mastering these architectures, engineers can assemble sophisticated systems that leverage the best capabilities of LLMs while mitigating their inherent limitations.

How to Use This Guide

Each section functions as a standalone reference but builds upon the others. Beginners should start with foundational patterns like *Prompt Chaining* and *Tool Use*. Advanced architects should focus on *Multi-Agent Collaboration* and *Resource-Aware Optimization*. Use the "Implementation Considerations" to weigh tradeoffs before writing code.

1. Prompt Chaining

Clear Definition

Prompt Chaining is the foundational design pattern where a complex task is decomposed into a series of smaller, sequential sub-tasks. Instead of asking an LLM to perform a massive operation in a single prompt (a "mega-prompt"), the output of one LLM call becomes the input for the next.

This approach transforms a probabilistic, opaque process into an assembly-line workflow. It allows for intermediate checkpoints where logic can be validated, formats can be corrected, and context can be refined before passing it to the next stage.

How It Works

- **Decomposition:** Break the user's request into logical steps (e.g., Step 1: Extract Data, Step 2: Analyze Sentiment, Step 3: Format Response).
- **Sequential Execution:** Execute Step 1. The output is captured in a variable.
- **Transformation:** (Optional) Code middleware cleans or reformats the output of Step 1.
- **Chaining:** The output of Step 1 is inserted into the prompt template for Step 2.
- **Validation:** Gates are placed between steps to ensure quality before proceeding.

Key Benefits

- **Higher Reliability:** Smaller tasks reduce the cognitive load on the model, leading to fewer errors.
- **Debuggability:** You can inspect the output at every stage to identify exactly where the failure occurred.
- **Context Management:** Prevents context window overflow by only passing necessary information to the next step.
- **Model Specialization:** Different steps can use different models (e.g., a fast model for formatting, a smart model for reasoning).

When to Use

Use Prompt Chaining for document processing pipelines (e.g., summarize -> translate -> email), complex content generation (outline -> draft -> critique -> refine), or any workflow where accuracy is more critical than latency.

Implementation Considerations

The primary tradeoff is latency; three sequential calls will always be slower than one parallel or single call. Cost also increases linearly with the number of tokens processed, although using smaller models for intermediate steps can mitigate this.

2. Routing

Clear Definition

Routing is a control flow pattern where a "Router" (or "Gateway") component analyzes an incoming request and directs it to the most appropriate downstream handler. This handler could be a specific prompt, a specialized agent, or a distinct model architecture.

Unlike a generalist agent that tries to do everything, a routing architecture acknowledges that specialization yields better results. It acts as a triage nurse, classifying intent (e.g., "This is a coding question" vs. "This is a creative writing request") and dispatching it accordingly.

How It Works

- **Input Analysis:** The user query is fed into a lightweight classifier (often a small LLM or even a keyword-based heuristic).
- **Intent Classification:** The router determines the category of the request (e.g., "Sales Inquiry", "Technical Support", "Refund").
- **Dispatch:** Based on the classification, the request is routed to a specific workflow or agent optimized for that intent.
- **Fallback:** If the intent is ambiguous, it routes to a generalist model or asks for clarification.

Key Benefits

- **Cost Efficiency:** Simple queries are routed to cheaper, faster models (e.g., GPT-3.5/Haiku), while complex ones go to SOTA models (e.g., GPT-4/Opus).
- **Specialization:** Allows you to curate prompts specifically for distinct domains, improving accuracy.
- **Separation of Concerns:** Keeps prompts clean and focused rather than having one prompt handling all logic.

Common Pitfalls

Over-routing can lead to maintenance nightmares. If you have 50 specific routes, managing them becomes difficult. Also, if the initial classification is wrong, the entire downstream process fails.

Real-World Examples

Customer Support Bot: A router detects if a user is asking for a password reset (routes to a deterministic API tool) or complaining about service (routes to an empathetic LLM agent).

3. Parallelization

Clear Definition

Parallelization allows an agent to perform multiple tasks simultaneously rather than sequentially. This is often implemented as a "Map-Reduce" or "Fan-out/Fan-in" architecture where a complex task is split into independent sub-tasks, processed in parallel, and then aggregated.

This pattern is essential for performance optimization in agentic workflows, significantly reducing wall-clock time for tasks that do not have strict sequential dependencies.

How It Works

- **Fan-Out:** The main agent identifies distinct sub-tasks (e.g., "Research competitor A", "Research competitor B", "Research competitor C").
- **Parallel Execution:** Multiple LLM calls are triggered asynchronously.
- **Normalization:** The outputs from these parallel calls are collected.
- **Fan-In (Aggregation):** A final "Synthesizer" step combines the varied outputs into a cohesive final response.

Key Benefits

- **Speed:** Drastically reduces total latency. 5 tasks taking 10 seconds each take 10 seconds total (plus overhead), not 50.
- **Diverse Perspectives:** You can run the same prompt through different personas simultaneously and aggregate the viewpoints (e.g., "Voting" mechanism).
- **Scalability:** Easily handles large datasets by chunking processing.

When to Use

Use when processing large documents (chunking), conducting broad market research, generating multiple creative variations for A/B testing, or implementing "elections" where multiple models vote on the best answer.

4. Reflection

Clear Definition

Reflection (also known as "Self-Correction" or "Critic Loops") is a pattern where an agent reviews its own output to identify errors, hallucinations, or areas for improvement before showing the result to the user. It mimics the human process of drafting and proofreading.

Instead of a single "shot" at the answer, the agent generates a draft, then switches to a "Critic" persona to analyze the draft against specific criteria, and finally generates a revised version.

How It Works

- **Generate:** The agent produces an initial draft response.
- **Critique:** A separate prompt (or the same model with a new instruction) is asked to review the draft. "Identify any logical fallacies or missing citations in the text above."
- **Evaluate:** If the critique finds issues, the feedback is passed back to the generator.
- **Revise:** The generator produces a new version incorporating the feedback.
- **Loop:** This can repeat for a set number of cycles or until quality criteria are met.

Implementation Considerations

Reflection increases cost and latency significantly (often 2x or 3x). It requires careful prompt engineering for the "Critic" role—if the critic is too lenient, the pattern is useless; if too pedantic, the loop may never resolve.

Pattern Combinations

Reflection is often combined with **Chain of Thought** reasoning. The agent "thinks aloud" about its errors. It is also crucial in **Coding Agents**, where the "Critic" step might actually involve running the code and reporting syntax errors.

5. Tool Use

Clear Definition

Tool Use (or "Function Calling") empowers LLMs to interact with the external world. Instead of relying solely on internal training data, the agent can call APIs, query databases, execute code, or browse the web.

This transforms the LLM from a text generator into a system controller. The LLM acts as the reasoning engine that decides *which* tool to use and *how* to format the inputs for that tool.

How It Works

- **Definition:** You provide the LLM with a schema describing available tools (e.g., ``calculator(a, b)``, ``search_web(query)``).
- **Selection:** The LLM analyzes the user prompt and decides it needs external data. It outputs a structured command (e.g., JSON) requesting a tool execution.
- **Execution:** The application runtime intercepts this request, executes the actual code/API, and gets the result.
- **Response:** The tool output is fed back to the LLM as context.
- **Final Answer:** The LLM incorporates the tool output to generate the final user response.

Key Benefits

- **Grounded Truth:** Eliminates hallucinations by retrieving real-time data (stock prices, weather).
- **Actionability:** Agents can actually *do* things (send emails, book meetings) rather than just talk about them.
- **Math & Logic:** Offloads complex calculations to Python/Calculators where LLMs struggle.

Common Pitfalls

Hallucinated Calls: The model might try to call a tool that doesn't exist or hallucinate parameters. Strict schema validation is required. **Infinite Loops:** An agent might keep calling a tool repeatedly if the output isn't what it expects.

6. Planning

Clear Definition

The Planning pattern involves an agent dynamically creating a multi-step plan to solve a complex goal before executing any actions. Unlike Prompt Chaining (which is a pre-defined rigid workflow), Planning is autonomous—the agent figures out the steps itself.

This allows agents to handle open-ended, ambiguous requests like "Plan a 2-week vacation to Japan" by breaking them down into dependencies, milestones, and required resources.

How It Works

- **Goal Analysis:** The agent receives a high-level goal.
- **Plan Generation:** The agent generates a structured list of steps (e.g., 1. Research flights, 2. Book hotels, 3. Create itinerary).
- **Dependency Mapping:** It identifies that Step 2 cannot happen before Step 1.
- **Execution & Tracking:** The agent executes steps one by one, marking them as complete.
- **Replanning:** If a step fails (e.g., "No flights found"), the agent updates the plan dynamically.

Key Benefits

- **Handling Ambiguity:** Enables completion of tasks that were not explicitly programmed by the developer.
- **Transparency:** Users can see the proposed plan and approve/modify it before execution starts.
- **Long-horizon Tasks:** Keeps the agent focused over long interactions without losing the thread.

Real-World Examples

DevOps Agent: "Migrate this database to AWS." The agent plans: 1. Audit current schema, 2. Spin up RDS instance, 3. Run migration script, 4. Verify data integrity.

7. Multi-Agent Collaboration

Clear Definition

This pattern involves multiple specialized agents working together to solve a problem, often orchestrated by a "Manager" agent. Instead of one agent trying to be a writer, coder, and reviewer, distinct agents assume these personas and communicate.

This mimics a human software team structure. It leverages the principle that specialized prompts perform better than general ones. The interaction can be hierarchical (Manager -> Workers) or conversational (Worker <-> Worker).

How It Works

- **Role Definition:** Define agents with specific system prompts (e.g., "You are a Python Expert", "You are a QA Engineer").
- **Task Assignment:** A Manager agent receives the user request and delegates sub-tasks to specific workers.
- **Shared State:** Agents share a message history or a "blackboard" where they post results.
- **Handoffs:** The Coder agent finishes code and passes it to the QA agent. The QA agent passes feedback back to the Coder.

Key Benefits

- **Modularity:** Easier to upgrade or fix one specific agent without breaking the whole system.
- **Higher Quality:** "Debate" between agents (e.g., a Red Team agent vs. a Blue Team agent) yields more robust solutions.
- **Context Optimization:** Each agent only needs the context relevant to its specific role.

When to Use

Use for complex creative projects (Game development: Story Agent, Art Agent, Music Agent) or enterprise workflows simulating departments (HR Agent, Finance Agent, Legal Agent).

8. Memory Management

Clear Definition

Memory Management patterns enable agents to retain information across different timescales. Without this, an agent resets after every session. This pattern structures memory into Short-term (context window), Episodic (recent history), and Long-term (knowledge base).

Effective memory management is crucial for personalization, learning from past mistakes, and maintaining continuity in long-running tasks.

How It Works

- **Short-Term:** Utilizing the immediate context window for the active conversation.
- **Episodic:** Storing summaries of recent interactions. When the context window fills, older messages are summarized and stored.
- **Long-Term (Semantic):** Using Vector Databases (RAG) to store facts and retrieve them based on similarity.
- **Procedural:** Storing "how-to" knowledge (e.g., "I learned that this user prefers concise Python code").

Implementation Considerations

Retrieval Strategy: The challenge is not storing data, but retrieving the *right* data at the right time. Retrieving too much irrelevant memory confuses the model ("Lost in the Middle" phenomenon).

Real-World Examples

Personal Tutor: Remembers a student's weak areas from a session three weeks ago (Long-term) and refers back to a question asked 5 minutes ago (Short-term).

9. Learning & Adaptation

Clear Definition

This pattern allows agents to improve their performance over time without model fine-tuning. It involves capturing feedback (from users or environment) and using it to update the agent's prompts, few-shot examples, or decision policies.

Instead of a static system, the agent builds a dynamic "knowledge file" of successful and unsuccessful strategies.

How It Works

- **Feedback Loop:** The agent executes a task. The user provides feedback (e.g., thumbs down, "Too verbose").
- **Insight Extraction:** The agent analyzes the feedback to derive a rule (e.g., "User X prefers bullet points").
- **Prompt Injection:** This rule is saved to a profile and injected into the system prompt for future interactions.
- **Optimization:** Frameworks like DSPy can mathematically optimize the prompts based on a dataset of successful examples.

Key Benefits

- **Personalization:** The agent adapts to specific user quirks.
- **Self-Healing:** If an agent keeps failing a specific API call, it learns to avoid that parameter.
- **Zero-Maintenance Improvement:** The system gets better the more it is used.

10. Goal Setting & Monitoring

Clear Definition

This pattern gives agents a "North Star." It involves explicitly defining Success Criteria (KPIs) and implementing a monitoring layer that checks if the agent is drifting from these goals during execution.

In autonomous loops, agents can easily get "distracted" or go down rabbit holes. This pattern acts as a supervisor, constantly asking "Are we closer to the goal?"

How It Works

- **Goal Definition:** The goal is formalized (e.g., "Find the cheapest flight under \$500").
- **State Tracking:** A "Monitor" process runs alongside the agent, analyzing the conversation state.
- **Drift Detection:** If the agent starts discussing hotels instead of flights, the Monitor triggers an intervention.
- **Course Correction:** The Monitor injects a system message: "You are deviating. Focus back on finding flights."

When to Use

Essential for autonomous agents that run for minutes or hours without human supervision, such as research agents or automated coding assistants.

11. Exception Handling & Recovery

Clear Definition

Robust agents must handle failures gracefully. This pattern defines standard procedures for when things go wrong: API timeouts, context limit errors, refusals (safety triggers), or nonsensical outputs.

It moves beyond "try/catch" in code to "cognitive recovery" where the agent understands *why* it failed and tries a different strategy.

How It Works

- **Classification:** Identify the error type. Is it transient (network)? Logical (bad code)? Or constraint (context length)?
- **Backoff Strategies:** For rate limits, implement exponential backoff.
- **Cognitive Retry:** If a tool use fails with "Invalid Argument," feed the error message back to the LLM so it can correct the argument and try again.
- **Graceful Degradation:** If the smart model (GPT-4) fails, fall back to a faster/simpler model or a manual rule-based path.

Key Benefits

- **Resilience:** Prevents the entire workflow from crashing due to one hiccup.
- **User Experience:** Instead of showing "Error 500", the agent says "I'm having trouble with that tool, let me try a different approach."

12. Human-in-the-Loop (HITL)

Clear Definition

HITL puts a human operator at critical decision points within the agent's workflow. The agent performs the heavy lifting (drafting, researching) but pauses for human approval before taking high-stakes actions.

This is critical for safety and trust in enterprise applications where AI autonomy cannot yet be fully trusted.

How It Works

- **Suspension:** The agent reaches a critical step (e.g., "Send Email to Client"). It suspends execution.
- **Presentation:** The draft email and context are presented to a human via UI.
- **Review:** The human can "Approve", "Edit", or "Reject".
- **Resumption:** The agent resumes execution based on the human's input. If edited, the agent learns from the change.

When to Use

Always use HITL for financial transactions, public-facing communications (social media posts), or deleting data. It bridges the gap between automation and safety.

13. Retrieval Augmented Generation (RAG)

Clear Definition

RAG is the standard pattern for connecting LLMs to private data. It retrieves relevant text chunks from a knowledge base and inserts them into the context window, allowing the model to answer questions about data it was never trained on.

While often treated as a standalone feature, in agentic systems, RAG is a dynamic tool that agents call on-demand.

How It Works

- **Ingestion:** Documents are parsed, split into chunks, and embedded into vectors.
- **Query Transformation:** The agent rewrites the user query to be optimized for search (e.g., removing conversational fluff).
- **Retrieval:** The system searches the vector DB for the most similar chunks.
- **Reranking:** A reranker model sorts the results by relevance.
- **Generation:** The top chunks are passed to the LLM to generate the answer.

Implementation Considerations

Chunking Strategy: How you split text matters. Too small = missing context; too large = noise. **Hybrid Search:** Combining vector search (semantic) with keyword search (BM25) often yields better results for specific terms.

14. Inter-Agent Communication

Clear Definition

As systems scale to Multi-Agent architectures, agents need a standardized protocol to talk to each other. This pattern defines the "language" of interaction: message formats, headers, handoff signals, and termination protocols.

It prevents the "Telephone Game" problem where context is lost or corrupted as it passes between agents.

How It Works

- **Protocol Definition:** JSON schemas are used for messages. `{ sender: "ResearchAgent", recipient: "WriterAgent", content: "...", context_summary: "..." }`.
- **Handshakes:** Agents explicitly acknowledge receipt of tasks.
- **Context Compression:** When passing a task, the sender compresses its internal state into a summary so the receiver doesn't have to read the full history.

Key Benefits

- **Decoupling:** Agents can be built by different teams or use different models as long as they adhere to the communication protocol.
- **Auditability:** Standardized logs make it easy to trace how a decision rippled through the network.

15. Resource-Aware Optimization

Clear Definition

This architectural pattern dynamically optimizes the trade-off between cost, latency, and quality. The system assesses the complexity of a task and allocates the minimum necessary computational resources to solve it.

It avoids the "using a sledgehammer to crack a nut" problem of sending every "Hello" message to GPT-4.

How It Works

- **Complexity Assessment:** A lightweight model scores the prompt's difficulty (1-10).
- **Model Selection:**
 - Score 1-3: Use local model (Llama-3-8b) or cheap API (GPT-3.5).
 - Score 4-7: Use mid-tier model (Claude Sonnet).
 - Score 8-10: Use flagship model (GPT-4/Opus).
- **Cascade:** Try the cheap model first. If the confidence score of the answer is low, escalate to the expensive model.

Key Benefits

Drastically reduces operational costs (often by 60-80%) while maintaining high quality for the difficult queries that actually matter.

16. Reasoning Techniques

Clear Definition

These are "thinking patterns" injected into the prompt structure to force the model to compute logically rather than intuitively. Examples include Chain-of-Thought (CoT), Tree-of-Thoughts (ToT), and Self-Consistency.

These techniques unlock the latent intelligence of models for math, logic, and planning tasks.

How It Works

- **Chain-of-Thought (CoT):** Appending "Let's think step by step" forces the model to output intermediate reasoning steps, which improves the final answer accuracy.
- **Tree-of-Thoughts (ToT):** The model generates multiple possible next steps, evaluates each, and explores the most promising branch (like a chess engine).
- **Self-Consistency:** Run the same CoT prompt 5 times and take the majority answer.

When to Use

Mandatory for math problems, riddles, complex debugging, or legal reasoning where the "path" to the answer is as important as the answer itself.

17. Evaluation & Monitoring (Evals)

Clear Definition

This pattern introduces rigorous software engineering testing to AI. It involves creating "Golden Sets" (test cases with known good answers) and automated scoring pipelines to measure agent performance before deployment.

Without Evals, you are flying blind, relying on "vibes" rather than data to judge if a prompt change improved or worsened the agent.

How It Works

- **Dataset Creation:** Curate 50-100 input/output pairs representing ideal behavior.
- **LLM-as-a-Judge:** Use a superior model (e.g., GPT-4) to grade the agent's output against the golden answer. Score on accuracy, tone, and conciseness.
- **Regression Testing:** Run these evals automatically in CI/CD pipelines whenever a prompt or code change is committed.

Key Benefits

- **Confidence:** Deploy changes knowing you haven't broken existing functionality.
- **Metric-Driven Development:** Move from "I think this is better" to "This improved accuracy by 4%."

18. Guardrails & Safety

Clear Definition

Guardrails are defensive layers that sit between the user and the agent, and the agent and the world. They enforce safety policies, prevent PII (Personally Identifiable Information) leakage, and block prompt injection attacks.

This pattern ensures the agent operates within ethical and legal boundaries, regardless of what the LLM generates.

How It Works

- **Input Guardrails:** Scan user input for malicious patterns (e.g., "Ignore previous instructions"). Block or sanitize if detected.
- **Output Guardrails:** Scan agent output for banned keywords, PII (emails, SSNs), or toxic content before sending to user.
- **Sandboxing:** Run code execution tools in isolated docker containers with no network access to prevent system compromise.

Real-World Examples

Healthcare Bot: A guardrail detects if a user mentions "suicide" and immediately overrides the LLM to provide crisis hotline numbers.

19. Prioritization

Clear Definition

In autonomous systems handling queues of tasks, not all tasks are equal. The Prioritization pattern uses an "Eisenhower Matrix" approach for agents to dynamically reorder their backlog based on Value, Effort, Urgency, and Risk.

This prevents agents from spending 2 hours on a low-value trivial task while a critical alert sits in the queue.

How It Works

- **Scoring:** When a task arrives, a "Triage Agent" scores it.
 - Value: (1-10) How important is this?
 - Effort: (1-10) Estimated token/time cost.
- **Ranking:** Tasks are sorted in the queue. High Value / Low Effort tasks are prioritized (Quick Wins).
- **Starvation Prevention:** Low priority tasks get a slight score boost over time so they are eventually processed.

Implementation Considerations

Requires a persistent queue system (like Redis or Celery) and a dedicated logic layer separate from the LLM execution.

20. Exploration & Discovery

Clear Definition

This pattern is used when the solution space is unknown. Instead of converging on an answer, the agent is designed to diverge—to map out the landscape, cluster information, and probe for novel insights.

It turns the agent into a researcher or scientist that generates hypotheses rather than just executing instructions.

How It Works

- **Divergent Generation:** The agent is prompted to generate 20 distinct ideas, explicitly maximizing for variance (high temperature).
- **Clustering:** Embed these ideas and cluster them to find themes.
- **Gap Analysis:** The agent looks at the clusters and asks "What is missing?" or "What is the whitespace?"
- **Probing:** It generates new queries to explore those gaps.

Key Benefits

- **Innovation:** Helps humans break out of writer's block or standard patterns of thinking.
- **Comprehensive Coverage:** Ensures a research agent doesn't just return the first 3 Google results but explores the entire topic.

Conclusion

Building Robust Systems

The transition from a "demo" to a "product" is defined by how well you implement these patterns. No single agent needs all 20, but every production system will need a combination of them. A typical robust architecture might look like:

- **Routing** to direct intent.
- **RAG** to ground knowledge.
- **Reasoning** (CoT) for logic.
- **Guardrails** for safety.
- **Evals** to measure success.

How to Start

Do not try to implement Multi-Agent Collaboration (Pattern 7) before you have mastered Prompt Chaining (Pattern 1) and Tool Use (Pattern 5). Start simple. Complexity adds failure modes. Add patterns only when a specific pain point (e.g., latency, hallucination, cost) demands it.

The future of Agentic AI is not just better models, but better engineering around those models. By mastering these patterns, you are future-proofing your skills and your systems against the rapid pace of AI evolution.

References & Further Reading

- **Source Material:** Based on the "Building Effective Agents" guide by Anthropic and Google DeepMind engineering practices.
- **Course:** "Master ALL 20 Agentic AI Design Patterns" by Mark Kashef (2025).
- **Academic Papers:** "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models" (Google Brain, 2022).
- **Frameworks:** LangChain, AutoGen, CrewAI, and DSPy documentation.